# Week-11

Karthik Thiagarajan

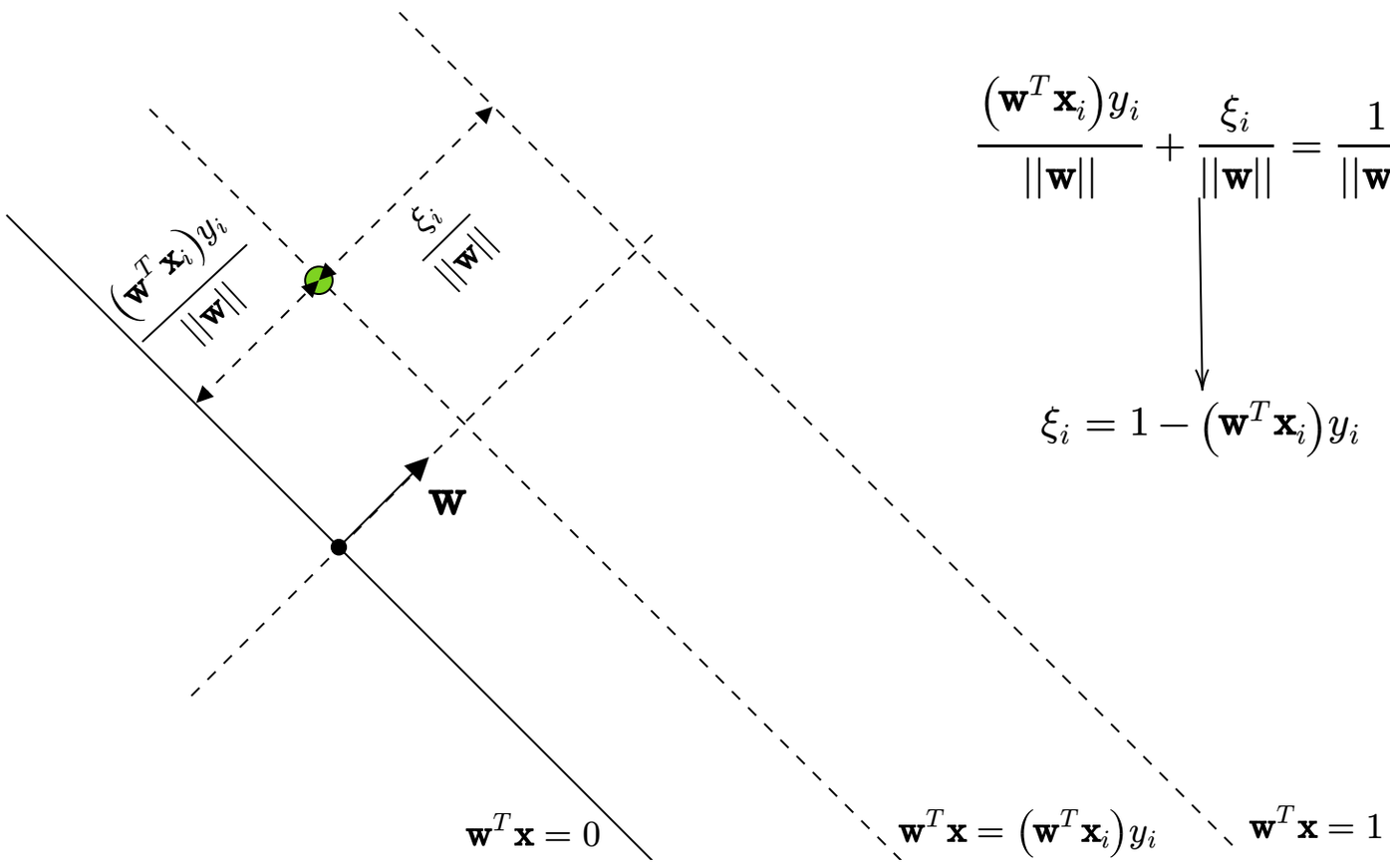

1. Soft Margin SVM ..... 2
   1.1. Soft Margin ..... 2
   1.2. Optimization Problem ..... 3
   1.3. Hinge Loss Formulation ..... 4
   1.4. Solving the Optimization Problem ..... 6
   1.5. Primal and Dual ..... 7
   1.6. Complementary Slackness and Support vectors ..... 8
      1.6.1. Primal view ..... 9
      1.6.2. Dual view ..... 11
   1.7. SVM Scenarios ..... 12
2. Overfitting and Underfitting ..... 13
3. Ensemble Techniques ..... 15
   3.1. Bagging ..... 15
      3.1.1. Bootstrapping ..... 15
      3.1.2. Aggregation ..... 15
      3.1.3. Distribution of points in a bag ..... 16
      3.1.4. Random Forest ..... 17
   3.2. Boosting ..... 17
      3.2.1. AdaBoost ..... 17

# 1. Soft Margin SVM

## 1.1. Soft Margin

We will get rid of the linear separability assumption. In the context of SVM, this would mean that we allow points to violate the margin. That is, we are not going to stipulate a strict inequality of the form $\left(\mathbf{w}^T\mathbf{x}_i\right)y_i \geqslant 1$. Instead, we are going to be more lenient and allow some slack. Hence, the origin of the name "soft-margin".

The two supporting hyperplanes continue to remain at $\mathbf{w}^T\mathbf{x} = \pm 1$. In order to penalize data-points that violate the margin, we assign a non-negative quantity $\xi_i$ for each data-point. Consider the specific case of a positive data-point violating the margin:

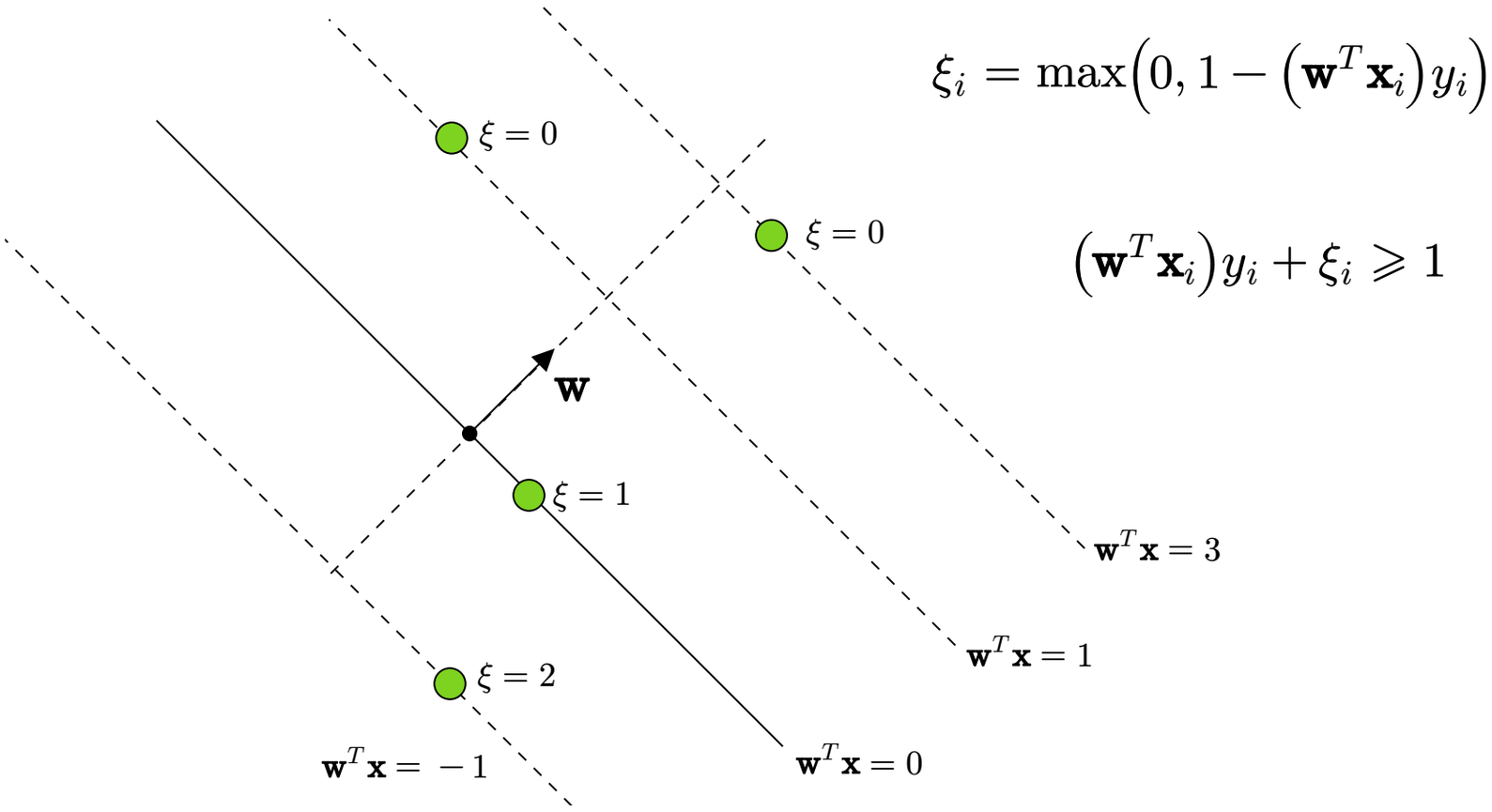



$$\frac{\left(\mathbf{w}^T\mathbf{x}_i\right)y_i}{||\mathbf{w}||} + \frac{\xi_i}{||\mathbf{w}||} = \frac{1}{||\mathbf{w}||}$$

$$\xi_i = 1 - \left(\mathbf{w}^T\mathbf{x}_i\right)y_i$$

We can think of $\xi_i$ as the minimum (scaled) distance that the point has to move

in the correct direction so that it no longer violates the margin. Here are some sample points along with their $\xi_i$ values:




Points that violate the margin and are farther away from the correct supporting hyperplane suffer greater penalty. Points that are beyond the margin and on the right side of it do not suffer any penalty. This is expressed in a neat formula:

$$\xi_i = \max\Big(0, 1 - \big(\mathbf{w}^T\mathbf{x}_i\big)y_i\Big)$$

## 1.2. Optimization Problem

We can now formulate the optimization problem as follows:

$$\min_{\mathbf{w},\boldsymbol{\xi}} \quad \frac{||\mathbf{w}||^2}{2} + C \cdot \sum_{i=1}^{n} \xi_i$$

subject to:

$$\begin{aligned} \left(\mathbf{w}^T\mathbf{x}_i\right)y_i + \xi_i &\geqslant 1 \\ \xi_i &\geqslant 0 \end{aligned}, \quad 1 \leqslant i \leqslant n$$

The objective has a new term $\sum_{i=1}^{n} \xi_i$ which is the total penalty paid across all data-points. $C$ is a hyperparameter that controls the trade-off between the width of the margin and the violations.

## 1.3. Hinge Loss Formulation

Since we know the exact expression for $\xi_i$, we can get rid of the constraints and move it into the objective:

$$\min_{\mathbf{w}} \ \frac{||\mathbf{w}||^2}{2} + C \cdot \sum_{i=1}^{n} \max\Big(0, 1 - \left(\mathbf{w}^T\mathbf{x}_i\right)y_i\Big)$$

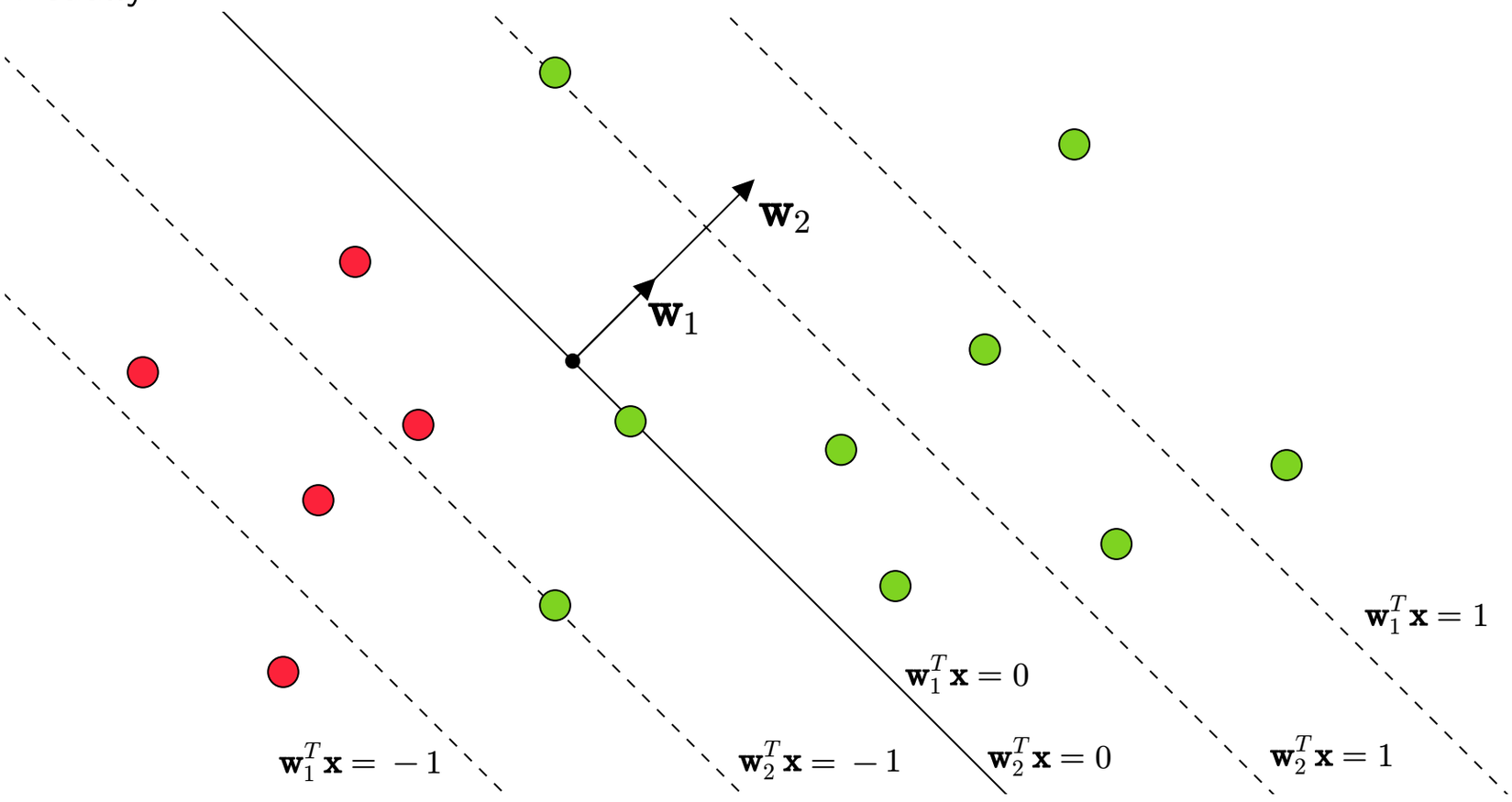
The expression $\sum_{i=1}^{n} \max\Big(0, 1 - \left(\mathbf{w}^T\mathbf{x}_i\right)y_i\Big)$ is termed the **hinge loss**. We can interpret the two terms in the loss as follows:

$$\underbrace{\frac{||\mathbf{w}||^2}{2}}_{\text{margin}} + \underbrace{C \cdot \sum_{i=1}^{n} \max\Big(0, 1 - \left(\mathbf{w}^T\mathbf{x}_i\right)y_i\Big)}_{\text{penalty}}$$

The first term controls the width of the margin, the second controls the total penalty levied on the data-points. These two terms work in opposite directions. That is:

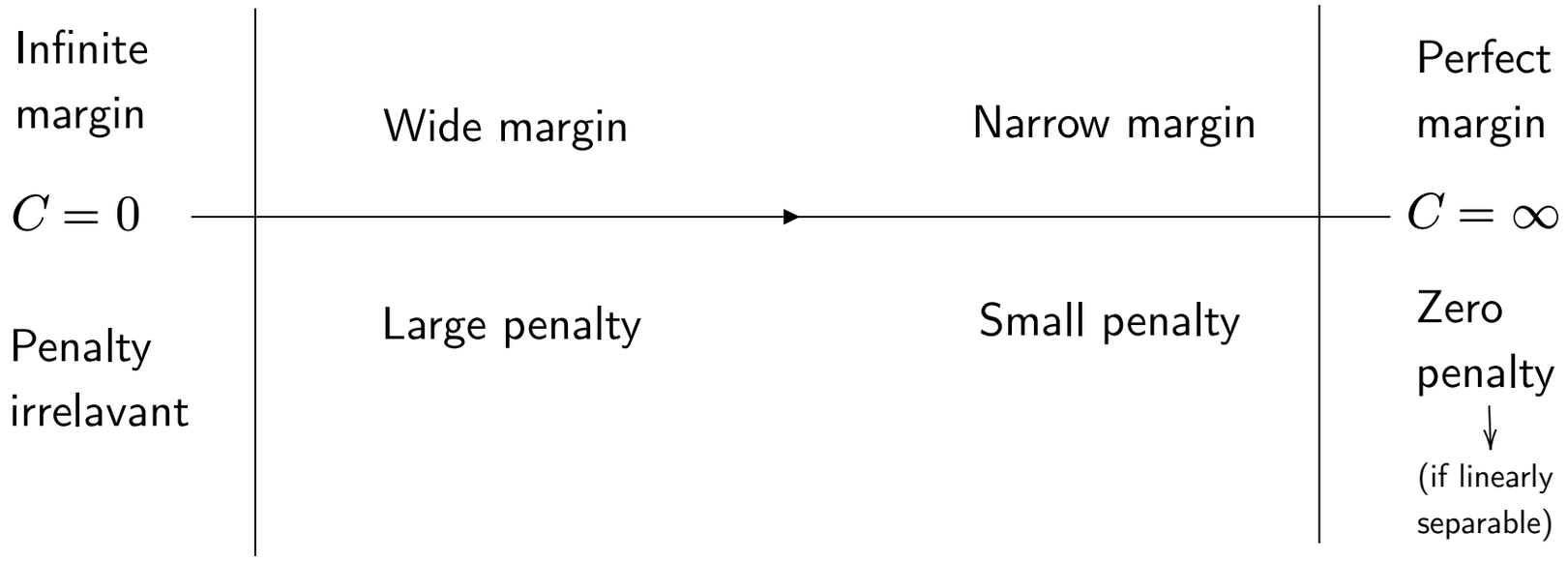
- If $||\mathbf{w}||$ is small, the margin is wide. If the margin is wide, the penalty paid by points will be generally high.
- If $||\mathbf{w}||$ is large, the margin is small. If the margin is small, the penalty paid by points will be generally low.

Visually:




- A very small value of $C$ implies that we don't mind the penalties, which encourages wide margins. In the limit as $C \to 0$, the supporting hyperplanes run away to infinity.
- For a very large $C$, we are very particular about small penalties, encouraging smaller margin. As $C \to \infty$, we approach hard-margin SVM.

Visually:




Alternatively, the soft-margin objective can be interpreted as the sum of a data-dependent loss and a regularization term ($L_2$). $\frac{1}{C}$ plays the role of the regularization rate.

$$\underbrace{\frac{||\mathbf{w}||^2}{2}}_{\text{Regularization}} + C \cdot \underbrace{\sum_{i=1}^{n} \max\Big(0, 1 - \big(\mathbf{w}^T \mathbf{x}_i\big) y_i\Big)}_{\text{Hinge Loss}}$$

## 1.4. Solving the Optimization Problem

We will revisit the optimization problem in its original form:

$$\min_{\mathbf{w}, \boldsymbol{\xi}} \quad \frac{||\mathbf{w}||^2}{2} + C \cdot \sum_{i=1}^{n} \xi_i$$

subject to:

$$\begin{aligned} \left(\mathbf{w}^T \mathbf{x}_i\right) y_i + \xi_i &\geqslant 1 \\ \xi_i &\geqslant 0 \end{aligned}, \quad 1 \leqslant i \leqslant n$$

Introducing the Lagrangian function and Lagrange multipliers, we get an equivalent formulation.

## 1.5. Primal and Dual

<u>Primal</u>

$$\min_{\mathbf{w},\boldsymbol{\xi}} \max_{\boldsymbol{\alpha}\geqslant 0,\boldsymbol{\beta}\geqslant 0} \frac{||\mathbf{w}||^2}{2} + C \cdot \sum_{i=1}^{n} \xi_i + \sum_{i=1}^{n} \alpha_i \left(1 - \left(w^T x_i\right) y_i - \xi_i\right) - \sum_{i=1}^{n} \beta_i \xi_i$$

<u>Dual</u>

$$\max_{\boldsymbol{\alpha}\geqslant 0,\boldsymbol{\beta}\geqslant 0} \min_{\mathbf{w},\boldsymbol{\xi}} \frac{||\mathbf{w}||^2}{2} + C \cdot \sum_{i=1}^{n} \xi_i + \sum_{i=1}^{n} \alpha_i \left(1 - \left(w^T x_i\right) y_i - \xi_i\right) - \sum_{i=1}^{n} \beta_i \xi_i$$

Solving the unconstrained optimization problem within the dual, we get:

$$\mathbf{w} = \sum_{i=1}^{n} \alpha_i \mathbf{x}_i y_i$$

and

$$\alpha_i + \beta_i = C, \quad 1 \leqslant i \leqslant n$$

As before, we can rewrite $\mathbf{w} = \mathbf{X}\mathbf{Y}\boldsymbol{\alpha}$. Since both $\alpha_i$ and $\beta_i$ are non-negative, we see that $\alpha_i, \beta_i \in [0, C]$ for all data-points.

The dual problem therefore becomes:

$$\max_{\mathbf{0} \leqslant \boldsymbol{\alpha} \leqslant C \cdot \mathbf{1}} \boldsymbol{\alpha}^T \mathbf{1} - \frac{1}{2} \boldsymbol{\alpha}^T \left( \mathbf{Y}^T \mathbf{X}^T \mathbf{X} \mathbf{Y} \right) \boldsymbol{\alpha}$$

This is nearly identical to the hard-margin form. The only change between the two forms is the constraints. As before, we won't solve the dual, but assume that there are solvers that would give us the optimal values for $\boldsymbol{\alpha}^*$.

## 1.6. Complementary Slackness and Support vectors

**Definition**
A support vector is a point for which $\alpha_i^* > 0$.

The complementary slackness conditions are:

$$\alpha_i^* \left[ 1 - \left( {\mathbf{w}^*}^T \mathbf{x}_i \right) y_i - \xi_i^* \right] = 0, \ 1 \leqslant i \leqslant n$$

$$\beta_i^* \xi_i^* = 0, \ 1 \leqslant i \leqslant n$$

Let us also add the primal and dual constraints:

<u>Primal constraints</u>

$$\xi_i^* \geqslant 0, \quad \left({\mathbf{w}^*}^T \mathbf{x}_i\right) y_i + \xi_i^* \geqslant 1$$

We also know:

$$\xi_i^* = \max\left(0, 1 - \left({\mathbf{w}^*}^T \mathbf{x}_i\right) y_i\right)$$

<u>Dual constraints</u>

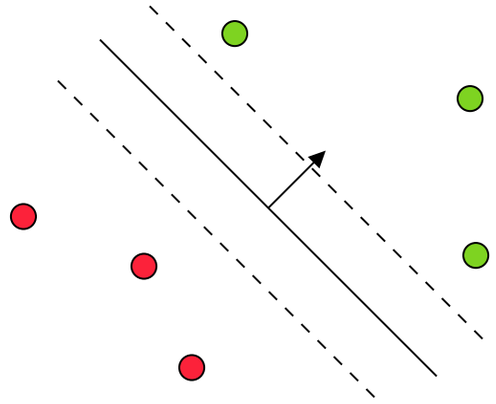
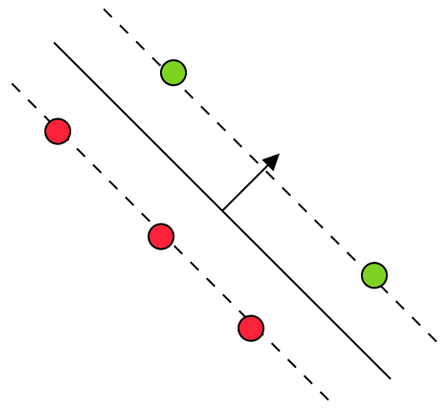
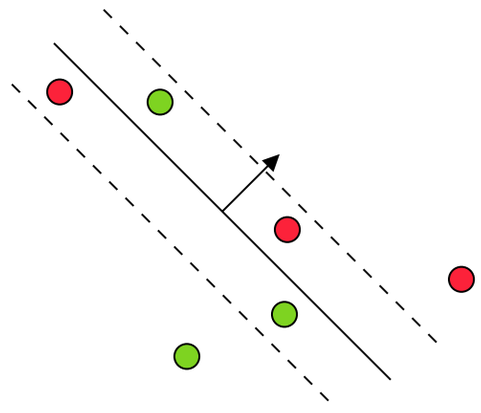
$$\alpha_i^* \geqslant 0, \quad \beta_i^* \geqslant 0, \quad \alpha^* + \beta_i^* = C$$

In the following tables, here is the naming convention:

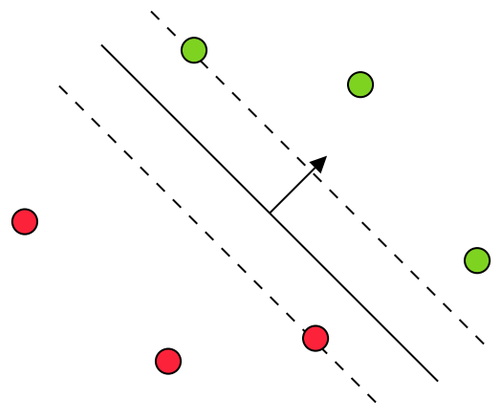
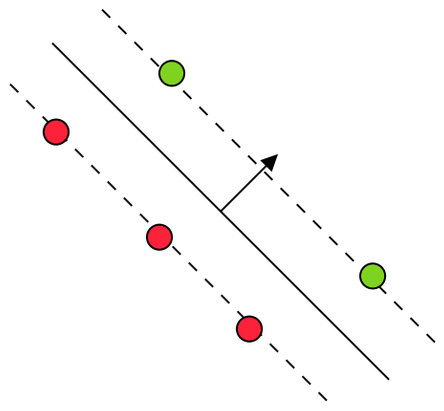
- Correct supporting hyperplane: $\mathbf{w}^T \mathbf{x} = 1$ for positive data-points and $\mathbf{w}^T \mathbf{x} = -1$ for negative data-points.
- Right side of the correct supporting hyperplane: $\mathbf{w}^T \mathbf{x} > 1$ for positive data-points and $\mathbf{w}^T \mathbf{x} < -1$ for negative data-points.
- Wrong side of the correct supporting hyperplane: $\mathbf{w}^T \mathbf{x} < 1$ for positive data-points and $\mathbf{w}^T \mathbf{x} > -1$ for negative data-points.

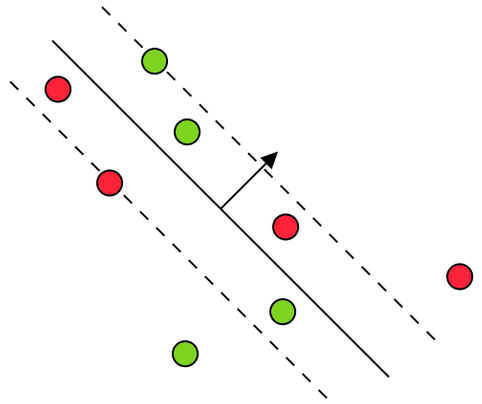
### 1.6.1. Primal view

| **Situation** | **Inference** | **Interpretation** |
|---|---|---|

| | | |
|---|---|---|
| $\left(\mathbf{w}^{*T}\mathbf{x}_i\right)y_i > 1$ | • $\xi_i^* = 0$<br>• $\alpha_i^* = 0$ | Points that lie on the right side of the correct supporting hyperplane are not support vectors.<br>  |
| $\left(\mathbf{w}^{*T}\mathbf{x}_i\right)y_i = 1$ | • $\xi_i^* = 0$<br>• $\alpha_i^* \in [0, C]$ | Points that lie on the correct supporting hyperplane *could* be support vectors.<br>  |
| $\left(\mathbf{w}^{*T}\mathbf{x}_i\right)y_i < 1$ | • $\xi_i^* > 0$<br>• $\alpha_i^* = C$ | Points that lie on the wrong side of the correct supporting hyperplane are some of the most important support vectors.<br>  |

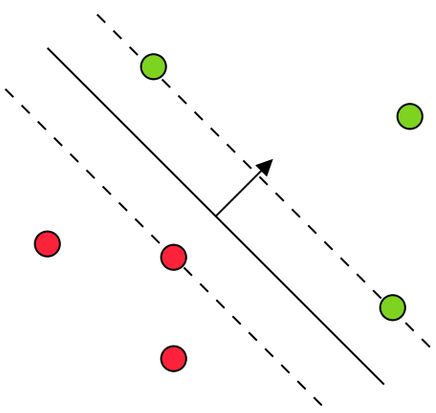
### 1.6.2. Dual view

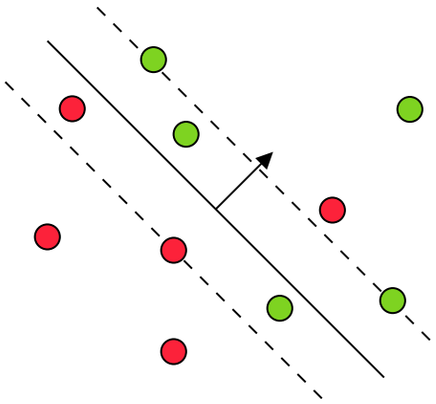
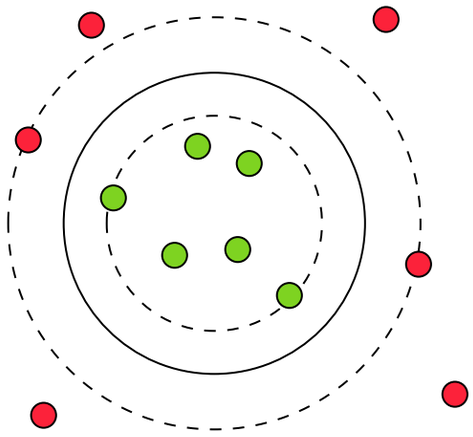
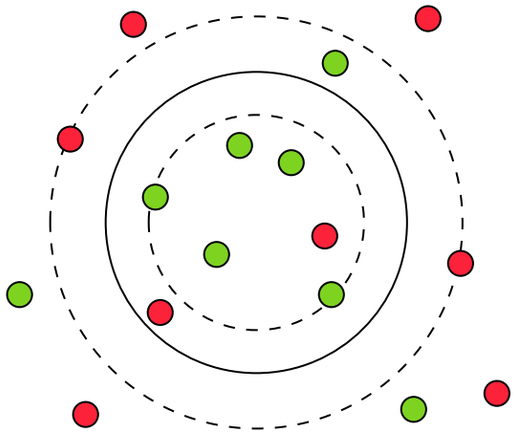
| **Situation** | **Inference** | **Interpretation** |
|---|---|---|
| $\alpha_i^* = 0$ | • $\xi_i^* = 0$<br>• $\left({\mathbf{w}^*}^T \mathbf{x}_i\right) y_i \geqslant 1$ | Points that are not support vectors lie on or on the right side of the correct supporting hyperplane.<br>  |
| $\alpha_i^* \in (0, C)$ | • $\xi_i^* = 0$<br>• $\left({\mathbf{w}^*}^T \mathbf{x}_i\right) y_i = 1$ | Points that are support vectors with $\alpha_i^* \neq C$ lie on the correct supporting hyperplanes.<br>  |

| | | |
|---|---|---|
| $\alpha_i^* = C$ | • $\xi_i^* \geqslant 0$<br>• $\left(\mathbf{w}^{*T}\mathbf{x}_i\right)y_i \leqslant 1$ | Points that are the most important support vectors may either lie on the correct supporting hyperplane or on the wrong side of it.<br>  |

## 1.7. SVM Scenarios

We have the following possibilities:

| **Type of Dataset** | **Type of SVM** |
|---|---|
|  | Hard-margin, Linear SVM |

| | |
|---|---|
|  | Soft-Margin, Linear SVM |
|  | Hard-Margin, Kernel SVM |
|  | Soft-Margin, Kernel SVM |

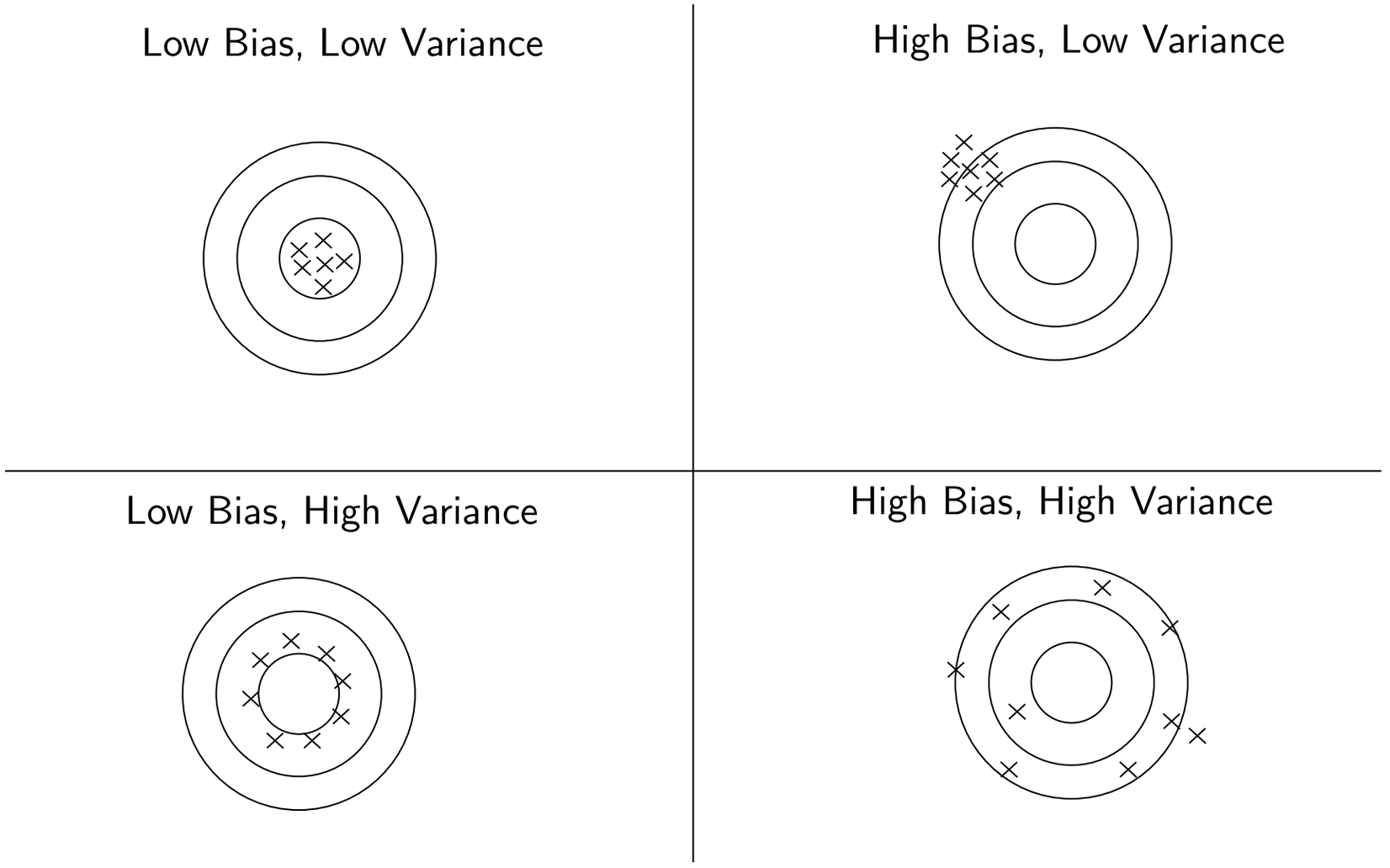
The kernel is also a hyperparameter.

## 2. Overfitting and Underfitting

- **Bias:** This measures the accuracy of the predictions or how close the model is to the true labels. Smaller the error, lower the bias. As the complexity of the model increases, bias tends to decrease. High bias models tend to underfit.

- **Variance:** This measures the variability in the model to fluctuations in the training dataset. As the complexity of the model increases, variance tends to increase. High variance models tend to overfit.

Bias and variance work in opposite directions. Models with a low bias tend to have high variance. Models with a high bias tend to have low variance.




Examples of low bias, high variance models:

- Deep decision trees
- Kernel regression with a polynomial kernel of high degree

Examples of high bias, low variance models:

- Decision stumps and very shallow decision trees
- Vanilla linear regression on a non-linear dataset

# 3. Ensemble Techniques

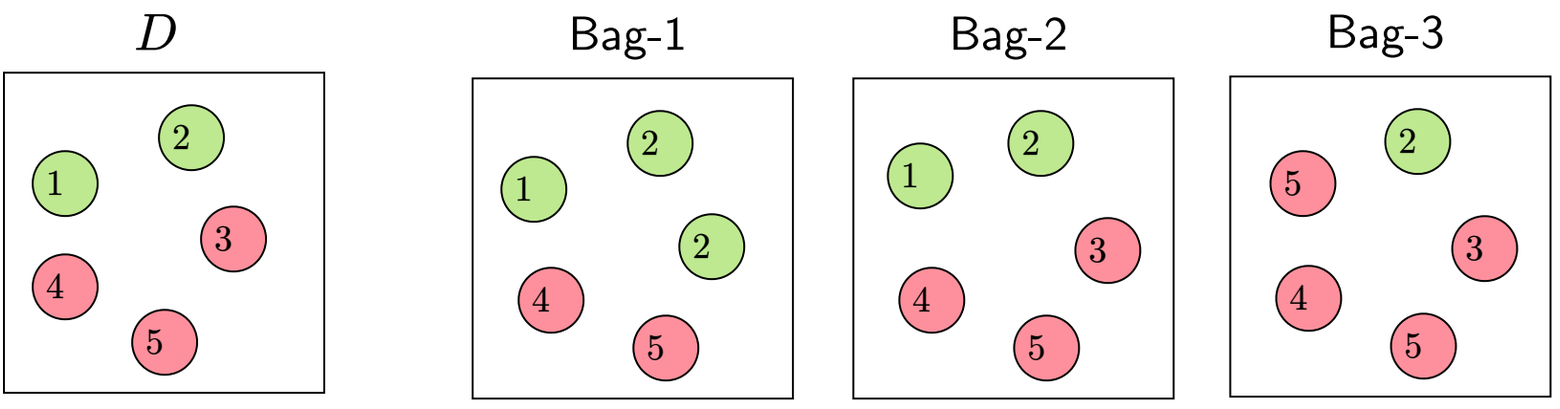
Ensemble techniques attempt to aggregate or combine multiple models to arrive at a decision.

## 3.1. Bagging

Bagging or bootstrap aggregation is a technique that tries to reduce the variance. The basic idea is this: averaging a set of observations reduces the variance. Instead of averaging observations, we will be averaging over models trained on different datasets. There are two steps:

### 3.1.1. Bootstrapping

We start with the dataset $D$ and create multiple versions or bags by sampling from $D$ with replacement. Each bag has the same number of data-points as $D$. Let us call the number of bags $B$. For example with $B = 3$:




Note that points can repeat in a given bag.

### 3.1.2. Aggregation

We train a model on each bag. Typically, this is a high-variance model, such as a deep decision tree. We call this $h_i$, the model for the $i^{th}$ bag. If there are $B$ bags, we can aggregate the outputs of all these models.

- For a regression problem, the method of aggregation could be the mean:

$$\frac{1}{B}\sum_{i=1}^{B} h_i(\mathbf{x})$$

- For a classification problem, the method of aggregation could be the majority vote:

$$\text{sign}\left(\sum_{i=1}^{B} h_i(\mathbf{x})\right)$$

### 3.1.3. Distribution of points in a bag

Fix a bag and consider an arbitrary data-point, say $(\mathbf{x}_i, y_i)$. The probability that this point gets picked as the first point in the bag is $\frac{1}{n}$. The probability that this point doesn't get picked as the first point is $1-\frac{1}{n}$. The probability that this point doesn't get picked at all in this bag is $\left(1-\frac{1}{n}\right)^n$, since we are sampling with replacement (independence). Therefore, the probability that this point appears at least once in a bag is (gets picked at all):

$$1-\left(1-\frac{1}{n}\right)^n$$

For a very large dataset, this is about $1-\frac{1}{e} \approx 63\%$. Therefore, about $63\%$ of

the data-points in a bag is unique.

### 3.1.4. Random Forest

Random forests bag deep decision trees with a slight twist. While growing each tree, at each split (question-answer pair), instead of considering all $d$ features, it randomly samples a subset of $m$ features to choose from. $m$ is typically around $\sqrt{d}$. In this way, the resulting trees are uncorrelated. Since the trees are grown independently on their own bags, the algorithm can be run in parallel.

## 3.2. Boosting

A weak learner is a model that does slightly better than a random model. For example, a model that produces an error rate of $0.48$ would be a weak model. An error rate of $0.48$ would mean that the model produces misclassifies $48\%$ of the data-point. Decision stump is an example of weak model.

In boosting, we combine weak learners, typically high bias models such as decision stumps, and turn them into strong learners, with a low bias. This is achieved by a sequential algorithm that focuses on mistakes committed at each step.

### 3.2.1. AdaBoost

**AdaBoost($D$, weak-learner)**

- $D_0(i) = \dfrac{1}{n},\ 1 \leqslant i \leqslant n$

- Repeat for $T$ rounds

  - $h_t \leftarrow$ Train a weak learner on $(D, D_{t-1})$

  - $\epsilon_t = \text{error}(h_t) = \displaystyle\sum_{h_t(x_i) \neq y_i} D_{t-1}(i)$

  - $\alpha_t = \dfrac{1}{2} \ln\left(\dfrac{1-\epsilon_t}{\epsilon_t}\right)$

  - $\widetilde{D}_t(i) = \begin{cases} e^{\alpha_t} \cdot D_{t-1}(i), & h_t(\mathbf{x}_i) \neq y_i, \quad \big(\text{mistake}\big) \\ e^{-\alpha_t} \cdot D_{t-1}(i), & h_t(\mathbf{x}_i) = y_i, \quad \big(\text{correct}\big) \end{cases}$

  - $D_t(i) = \dfrac{\widetilde{D}_t(i)}{\sum_{i=1}^{n} \widetilde{D}_t(i)}$

- Return $\text{sign}\left(\displaystyle\sum_{t=1}^{T} \alpha_t \cdot h_t\right)$

Observations:

- Mistakes are boosted by $e^{\alpha_t}$ and correct data-points are brought down by

$e^{-\alpha_t}$.

- If $\epsilon_t$ is high, then $\alpha_t$ is low. That is, if the classifier in round $t$ has a higher error, it is assigned a lower weight in the final ensemble.

It can be shown that as the number of rounds increases, the training error keeps decreasing. The training error of the final classifier after $T$ rounds is at most:

$$\exp\left(-2\sum_{t=1}^{T}\gamma_t^2\right)$$

where $\gamma_t = \dfrac{1}{2} - \epsilon_t$.